Cross-Residual Graph Neural Networks for Protein-Oriented Graph Classification

Xuelin Hu[1] **and Xiaoqin Fu**[2]

[1]**School of Railway Communication and Signaling Engineering, Liuzhou Railway Vocational Technical College, Liuzhou 545000, China**
[2]**School of Railway Locomotive and Rolling Stock, Liuzhou Railway Vocational Technical College, Liuzhou 545000, China**

Corresponding author:
Xiaoqin Fu[2]

Email address: fuxiaoqin@ltzy.edu.cn

ABSTRACT

Background. Existing graph neural networks often use skip connections to stabilize deep message passing, but they rarely isolate whether cross-branch exchange or graph-summary reinjection is preferable to ordinary same-path residual reuse in protein-oriented graph classification.
Methods. We propose Cross-Residual Graph Neural Networks (CR-GNN), a controlled comparison framework spanning five information-flow topologies: plain stacking, node-level residual reuse, node-level cross exchange, graph-level residual propagation, and graph-level cross-summary exchange. All variants are evaluated under shared backbones and stratified five-fold validation. The main evidence comes from the protein-oriented pair PROTEINS and DD; ENZYMES is treated as a supporting stress test, and MUTAG, AIDS, and Mutagenicity as supplementary robustness datasets. Targeted ablations and depth-scaling analyses are used to interpret the benchmark patterns.
Results. No single reuse topology dominates. In the main protein-oriented comparison, NodeResGNN is best on PROTEINS and GraphResGNN is best on DD. ENZYMES points in the same direction as GraphResGNN, but its low absolute accuracy makes it better interpreted as a stress test than as primary biological evidence. Cross-residual variants are not universal winners, but they remain competitive on DD and pair well with sparse routing in targeted ablations. In the 24 dataset–operator winner count, GraphResGNN records the most wins (12/24). A depth-scaling analysis on PROTEINS further shows a CKA–performance paradox: GraphResGNN has the lowest deepest-layer CKA yet the most wins, whereas NodeResGNN has the highest CKA but far fewer wins.
Conclusions. Residual design is dataset-dependent in protein-oriented graph classification. Graph-level residual propagation is the strongest default choice in this benchmark, but it is not uniformly best; node-level residual reuse remains strongest on selected datasets, and cross-residual exchange is a selective alternative rather than a universal replacement. Stronger layer-wise continuity also does not necessarily predict better final accuracy.

1 INTRODUCTION

Graph neural networks (GNNs) are now a standard approach for learning from molecular and protein graphs because they encode local neighborhood structure together with higher-order relational context (Kipf and Welling, 2016; Gilmer et al., 2017; Hamilton et al., 2017; Wu et al., 2021; Zhou et al., 2020). In biomolecular graph classification, this combination matters because predictive evidence may depend simultaneously on local biochemical motifs and broader graph-level organization. The present paper focuses on the protein-oriented benchmark setting represented most directly by PROTEINS and DD, using ENZYMES as a supporting biological stress test and treating MUTAG, AIDS, and Mutagenicity as supplementary robustness datasets rather than as equally central biological evidence. The goal is therefore not to claim direct validation for downstream biological discovery, but to study controlled residual-routing behavior in protein-related graph classification settings.

The main methodological difficulty is that deeper message passing is not automatically beneficial in these settings. Repeated propagation can suppress informative intermediate states through over-smoothing

and related depth effects (Li et al., 2018; Oono and Suzuki, 2020; Chen et al., 2020b), while over-squashing can cause information from distant nodes to be compressed into uninformative bottlenecks (Alon and Yahav, 2021; Topping et al., 2021). Residual connections, normalization, and regularization alleviate some of this degradation (He et al., 2016; Bresson and Laurent, 2017; Zhao and Akoglu, 2020; Cai et al., 2021; Rong et al., 2019; Chen et al., 2020a), but most residual designs still reuse information along the same branch. As a result, they improve optimization without directly testing whether alternative information-flow topologies preserve more useful historical signals.

Related graph-learning work has already explored skip connections, dense aggregation, APPNP-style propagation, Jumping Knowledge readout, and hierarchical pooling (Xu et al., 2018; Klicpera et al., 2018; Gao and Ji, 2019; Ying et al., 2018). These methods preserve information from multiple depths or across pooling hierarchies, yet they do not directly isolate whether exchanging hidden states across branches or reinjecting pooled graph summaries across block boundaries is preferable to ordinary same-path residual reuse. No prior study directly compares these reuse topologies under a controlled protocol—this is the comparison the present work provides.

This paper studies Cross-Residual Graph Neural Networks (CR-GNN), a controlled model family that compares plain stacking, node-level residual reuse, node-level cross-residual exchange, graph-level residual propagation, and graph-level cross-summary exchange under shared graph-classification backbones. The main paper-facing evidence is drawn from the protein-oriented core, especially PROTEINS and DD, while the non-protein datasets are used only to test whether the same architectural preferences remain stable outside that core. The study is organized around three specific research questions:

1. **Why can the architecture with the weakest representational continuity still win most often?** If residual reuse is commonly motivated as a way to preserve useful information across depth, what does it mean when the best-performing architecture instead shows the largest layer-wise representational change?

2. **When does each reuse topology dominate?** Across the main protein-oriented comparison defined by PROTEINS and DD, together with supporting stress-test and supplementary robustness datasets, when is node-level residual reuse the strongest choice, when is graph-level residual propagation preferable, and when does cross-residual exchange become a competitive or preferred alternative?

3. **How does residual routing design change the answer?** Beyond the binary question of whether an extra reuse path exists, how do different filtering strategies—learnable dense routing, top-k selection, and sparse suppression—affect the relative performance of residual and cross-residual architectures?

Framing the study around these three questions is important for the paper's methodological scope. Rather than claiming that one additional pathway always yields a better GNN, the goal is to reveal *when and how* residual routing helps, when ordinary residual reuse remains stronger, and why layer-wise continuity does not map monotonically to final accuracy once backbone, data split, and training protocol are controlled.

The work is organized as a protein-oriented biomolecular graph-classification methods study. The main evidence comes from PROTEINS and DD, with ENZYMES retained as a harder supporting biological benchmark whose poor absolute accuracy is itself informative about task difficulty. MUTAG, AIDS, and Mutagenicity are kept to test robustness outside the protein-centered core, but they do not define the main biological claim. This design keeps the paper close to biologically meaningful graph settings while maintaining a reproducible and controlled benchmark scope.

1.1 Contributions

The main contributions of this work are:

- **A controlled comparison framework for reuse topologies in protein-oriented graph classification.** Five information-flow designs—plain stacking, node-level residual reuse, node-level cross-residual exchange, graph-level residual propagation, and graph-level cross-summary exchange—are compared under identical backbones, data splits, and training protocols. By holding all other factors constant, the framework isolates the effect of reuse topology on performance, consistency, and representational dynamics, with the strongest biological emphasis placed on PROTEINS and DD.

- **The identification of systematic trade-offs between peak accuracy, rank stability, and representational continuity.** No single topology dominates all three dimensions. In the current benchmark, graph-level residual propagation achieves the highest peak accuracy most often but with noticeable rank volatility, cross-residual variants remain competitive and often stable without being universal winners, and the strongest representational continuity does not predict the best classification accuracy. This three-way tension, quantified through winner-count and rank-stability metrics, provides a new lens for evaluating GNN architectures beyond single-number leaderboard comparisons.

- **A depth-scaling mechanism analysis revealing a CKA–performance paradox.** Layer-wise CKA, cosine similarity, and gradient norm tracking across depths shows that the architecture with the weakest representational continuity achieves the most frequent wins—challenging the assumption that stronger layer-wise stability predicts better performance. This finding suggests that structured representational change, when coupled with appropriate context reinjection, can be productive rather than destructive.

- **Evidence that routing strategy is a meaningful secondary design axis.** Mild sparse routing is the most frequent winner across representative targets, especially on cross-oriented settings, whereas learnable dense routing remains strongest on the best graph-level residual target. This reframes the discussion from binary architectural choices to the joint selection of topology and routing strategy.

2 MATERIALS AND METHODS

This section defines the common graph-classification setting, the CR-GNN architecture family, the benchmark suite, and the shared experimental protocol. The guiding principle is to isolate where information is reused while keeping the remaining pipeline as consistent as possible. That design choice is central to the paper's methodological justification: if backbone operator, pooling, and evaluation protocol are held fixed, then performance differences are more plausibly attributable to the reuse topology itself rather than to unrelated implementation changes.

2.1 Problem definition and notation

Let $\mathcal{G} = (\mathcal{V}, \mathcal{E})$ denote an undirected or directed graph, where $\mathcal{V} = \{v_1, v_2, \ldots, v_n\}$ is the set of n nodes and $\mathcal{E} \subseteq \mathcal{V} \times \mathcal{V}$ is the set of edges. Each node $v_i \in \mathcal{V}$ is associated with a feature vector $\mathbf{x}_i \in \mathbb{R}^{d_{\text{in}}}$, and the node features for the entire graph are collected in $\mathbf{X} \in \mathbb{R}^{n \times d_{\text{in}}}$. The graph structure is represented using an adjacency matrix $\mathbf{A}$.

Given a training dataset $\mathcal{D} = \{(\mathcal{G}_i, y_i)\}_{i=1}^{N}$, the objective is to learn a mapping function $f_\theta : \mathcal{G} \to \hat{y}$ that predicts the label of an unseen graph. The present study compares five architectures under the same graph-classification pipeline:

- **PlainGNN**: sequential message passing without explicit state reuse
- **NodeResGNN**: single-branch node-level residual reuse
- **NodeCrossGNN**: two-branch node-level cross exchange
- **GraphResGNN**: graph-level residual propagation across sequential blocks
- **GraphCrossGNN**: graph-level cross-summary exchange across paired blocks

The comparison is designed to isolate where information is reused: inside one branch, across parallel branches, or through pooled graph summaries passed between blocks. This isolates a concrete research question behind the architecture family. Same-branch residual reuse mainly tests whether re-injecting historical states stabilizes deeper propagation, whereas cross-residual designs test whether exchanging information across parallel streams can preserve complementary structure that would otherwise be lost under ordinary stacking.

2.2 Shared message passing and graph-level learning

Let Φ denote a message-passing operator. For a chosen backbone, node representations are updated over L layers as

$$\mathbf{h}_i^{(\ell+1)} = \sigma\left(\mathbf{W}^{(\ell)} \cdot \mathrm{AGGREGATE}_\Phi\left(\left\{\mathbf{h}_j^{(\ell)} : j \in \mathscr{N}(i)\right\}, \mathbf{h}_i^{(\ell)}\right) + \mathbf{b}^{(\ell)}\right). \tag{1}$$

After L layers, a readout function R aggregates node-level representations into a graph embedding

$$\mathbf{h}_{\mathscr{G}} = R\left(\left\{\mathbf{h}_i^{(L)} : i = 1, \ldots, n\right\}\right). \tag{2}$$

The classifier produces

$$\hat{\mathbf{y}} = \mathrm{softmax}\left(\mathbf{W}_{\mathrm{clf}}\mathbf{h}_{\mathscr{G}} + \mathbf{b}_{\mathrm{clf}}\right), \tag{3}$$

and the model is trained end-to-end by minimizing cross-entropy:

$$\mathscr{L}(\theta) = -\frac{1}{N}\sum_{i=1}^{N}\sum_{c=1}^{C} y_{i,c}\log(\hat{y}_{i,c}). \tag{4}$$

2.3 CR-GNN architecture family

CR-GNN is defined as a compact family of graph classification architectures. All variants share the same input projection, stacked message-passing layers, graph-level pooling, and linear classifier; they differ only in how intermediate node states and graph-level summaries are reused across depth and across branches. The design deliberately avoids introducing separate encoders, task-specific feature engineering, or architecture-specific readout heads, because such additions would make it harder to interpret whether any observed gain comes from residual topology or from extra model capacity.

Let $\mathscr{G} = (\mathscr{V}, \mathscr{E})$ be a graph with node feature matrix $\mathbf{X} \in \mathbb{R}^{n \times d_{\mathrm{in}}}$ and adjacency structure $\mathbf{A}$. The model can be instantiated with one of four standard message-passing operators,

$$\Phi \in \{\mathrm{GCNConv},\ \mathrm{GATConv},\ \mathrm{SAGEConv},\ \mathrm{GINConv}\},$$

and reuses the same operator type throughout a model instance. For a hidden width d, every branch starts with an input projection

$$\mathbf{H}^{(0)} = \psi(\Phi_{\mathrm{in}}(\mathbf{X}, \mathbf{A})), \tag{5}$$

followed by L hidden layers and a readout

$$\mathbf{g} = \mathrm{Pool}\left(\mathbf{H}^{(L)}\right). \tag{6}$$

Here $\Phi_{\mathrm{in}} : \mathbb{R}^{n \times d_{\mathrm{in}}} \to \mathbb{R}^{n \times d}$ denotes the branch-specific input operator, $\mathrm{Pool}(\cdot)$ is global mean pooling over nodes, and

$$\psi(\mathbf{Z}) = \mathrm{Dropout}(\mathrm{ReLU}(\mathbf{Z})). \tag{7}$$

Residual signals are modulated by a scalar gate $\alpha \in (0, 1]$, learned by sigmoid parameterization in the default setting and fixed only in gate ablations. The residual-routing operator is written as

$$\mathscr{R}(\mathbf{U}) = \begin{cases} \mathbf{U}, & \text{learnable dense routing,} \\ \mathbf{U} \odot \mathbf{M}_k, & \text{top-k routing,} \\ \mathrm{softshrink}_\lambda(\mathbf{U}), & \text{sparse routing,} \end{cases} \tag{8}$$

where $\mathbf{M}_k$ keeps the largest-magnitude channels along the feature dimension and λ is the sparse-routing threshold.

2.3.1 Single-branch and node-level residual variants

PlainGNN. The plain baseline stacks L copies of the same operator without residual reuse:

$$\mathbf{H}^{(\ell+1)} = \psi\Big(\Phi_\ell\Big(\mathbf{H}^{(\ell)}, \mathbf{A}\Big)\Big), \qquad \ell = 0, \ldots, L-1. \tag{9}$$

NodeResGNN. The first residual variant stores the previous hidden state within the same branch. To match the implementation, the first residual input is zero:

$$\mathbf{P}^{(0)} = \mathbf{0}, \qquad \mathbf{P}^{(\ell+1)} = \mathbf{H}^{(\ell)}. \tag{10}$$

The update is

$$\mathbf{H}^{(\ell+1)} = \psi\Big(\Phi_\ell\Big(\mathbf{H}^{(\ell)} + \alpha\mathscr{R}\Big(\mathbf{P}^{(\ell)}\Big), \mathbf{A}\Big)\Big), \qquad \ell = 0, \ldots, L-1. \tag{11}$$

NodeCrossGNN. The node-level cross-residual model maintains two parallel branches with separate input projections:

$$\mathbf{H}_1^{(0)} = \psi\Big(\Phi_{\text{in}}^{(1)}(\mathbf{X}, \mathbf{A})\Big), \qquad \mathbf{H}_2^{(0)} = \psi\Big(\Phi_{\text{in}}^{(2)}(\mathbf{X}, \mathbf{A})\Big), \tag{12}$$

with zero-initialized exchanged states

$$\mathbf{P}_1^{(0)} = \mathbf{0}, \qquad \mathbf{P}_2^{(0)} = \mathbf{0}, \qquad \mathbf{P}_b^{(\ell+1)} = \mathbf{H}_b^{(\ell)}. \tag{13}$$

Each branch receives the previous state of the other branch:

$$\mathbf{H}_1^{(\ell+1)} = \psi\Big(\Phi_\ell^{(1)}\Big(\mathbf{H}_1^{(\ell)} + \alpha\mathscr{R}\Big(\mathbf{P}_2^{(\ell)}\Big), \mathbf{A}\Big)\Big), \tag{14}$$

$$\mathbf{H}_2^{(\ell+1)} = \psi\Big(\Phi_\ell^{(2)}\Big(\mathbf{H}_2^{(\ell)} + \alpha\mathscr{R}\Big(\mathbf{P}_1^{(\ell)}\Big), \mathbf{A}\Big)\Big), \qquad \ell = 0, \ldots, L-1. \tag{15}$$

The final node representation is the branch sum

$$\mathbf{H}^{(L)} = \mathbf{H}_1^{(L)} + \mathbf{H}_2^{(L)}. \tag{16}$$

2.3.2 Graph-level residual propagation

GraphResGNN. This variant chains three graph-conditioned blocks. Let $\mathscr{B}(\mathbf{X}, \mathbf{A}, \mathbf{g})$ denote one graph-conditioned block. Inside the block, a graph-level state $\mathbf{g} \in \mathbb{R}^{B\times d}$ is broadcast back to nodes by batch indexing and added before each hidden message-passing layer:

$$\mathbf{Z}^{(0)} = \psi(\Phi_{\text{in}}(\mathbf{X}, \mathbf{A})), \tag{17}$$

$$\mathbf{Z}^{(\ell+1)} = \psi\Big(\Phi_\ell\Big(\mathbf{Z}^{(\ell)} + \alpha\mathscr{R}(\mathbf{g}[\text{batch}]), \mathbf{A}\Big)\Big), \qquad \ell = 0, \ldots, L-1, \tag{18}$$

$$\mathscr{B}(\mathbf{X}, \mathbf{A}, \mathbf{g}) = \text{Pool}\Big(\mathbf{Z}^{(L)}\Big) + \alpha\mathscr{R}(\mathbf{g}). \tag{19}$$

If no previous graph state is available, the block uses $\mathbf{g} = \mathbf{0}$. GraphResGNN then applies the recurrence

$$\mathbf{g}^{(0)} = \mathbf{0}, \qquad \mathbf{g}^{(t)} = \mathscr{B}_t\Big(\mathbf{X}, \mathbf{A}, \mathbf{g}^{(t-1)}\Big), \qquad t = 1, 2, 3, \tag{20}$$

and uses $\mathbf{g}^{(3)}$ for classification.

GraphCrossGNN. Four blocks are organized as two sequential pairs. In this model, each block uses graph-level exchange only at the pooled embedding stage, not inside node updates. Let $\mathscr{C}(\mathbf{X}, \mathbf{A}, \mathbf{g})$ denote such a block:

$$\mathbf{Z}^{(0)} = \psi(\Phi_{\text{in}}(\mathbf{X}, \mathbf{A})), \tag{21}$$

$$\mathbf{Z}^{(\ell+1)} = \psi\Big(\Phi_\ell\Big(\mathbf{Z}^{(\ell)}, \mathbf{A}\Big)\Big), \qquad \ell = 0, \ldots, L-1, \tag{22}$$

$$\mathscr{C}(\mathbf{X}, \mathbf{A}, \mathbf{g}) = \text{Pool}\Big(\mathbf{Z}^{(L)}\Big) + \alpha\mathscr{R}(\mathbf{g}). \tag{23}$$

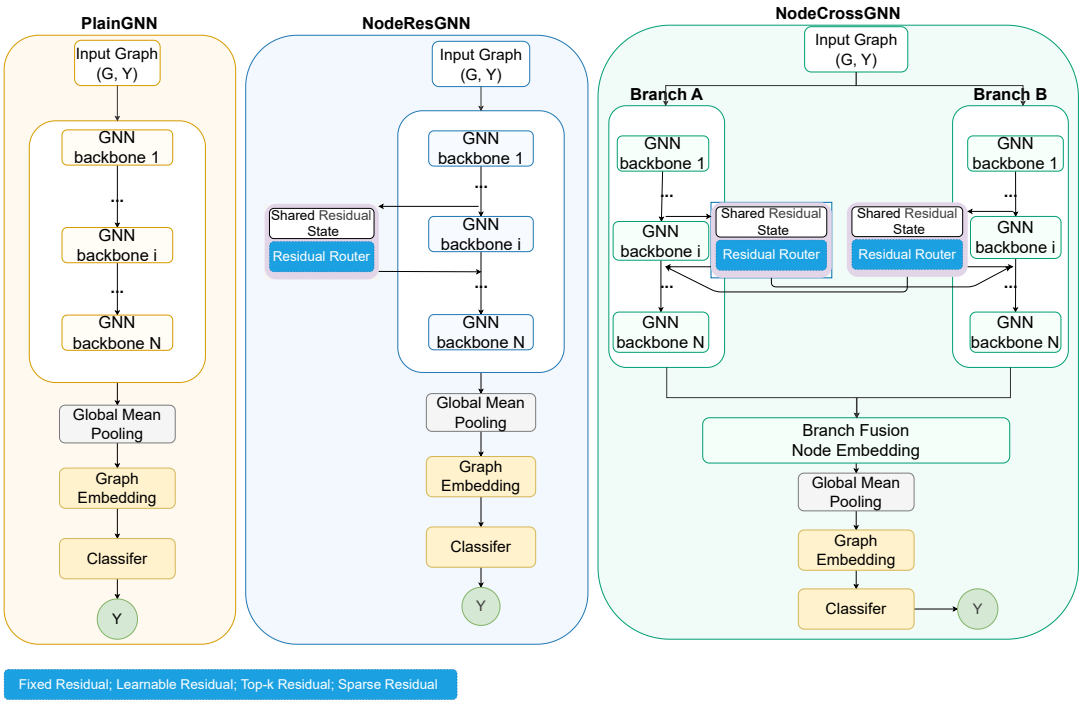

Figure 1. Node-level CR-GNN architecture family. Left: PlainGNN — sequential message-passing layers without state reuse. Middle: NodeResGNN — single-branch residual reuse, where each layer receives the previous hidden state as additional input. Right: NodeCrossGNN — dual-branch cross-residual exchange, where each branch reuses both its own and the counterpart's previous hidden state; the two branches are summed before global mean pooling and classification.

With two branch states initialized as $\mathbf{g}_1^{(0)} = \mathbf{g}_2^{(0)} = \mathbf{0}$, the stage-wise recurrence is

$$\widehat{\mathbf{g}}_1^{(t)} = \mathscr{C}_{2t-1}\left(\mathbf{X}, \mathbf{A}, \mathbf{g}_1^{(t-1)}\right), \tag{24}$$

$$\widehat{\mathbf{g}}_2^{(t)} = \mathscr{C}_{2t}\left(\mathbf{X}, \mathbf{A}, \mathbf{g}_2^{(t-1)}\right), \qquad t = 1, 2, \tag{25}$$

$$\mathbf{g}_1^{(t)} = \widehat{\mathbf{g}}_2^{(t)}, \qquad \mathbf{g}_2^{(t)} = \widehat{\mathbf{g}}_1^{(t)}. \tag{26}$$

The final graph embedding is

$$\mathbf{g}_{\text{final}} = \mathbf{g}_1^{(2)} + \mathbf{g}_2^{(2)}. \tag{27}$$

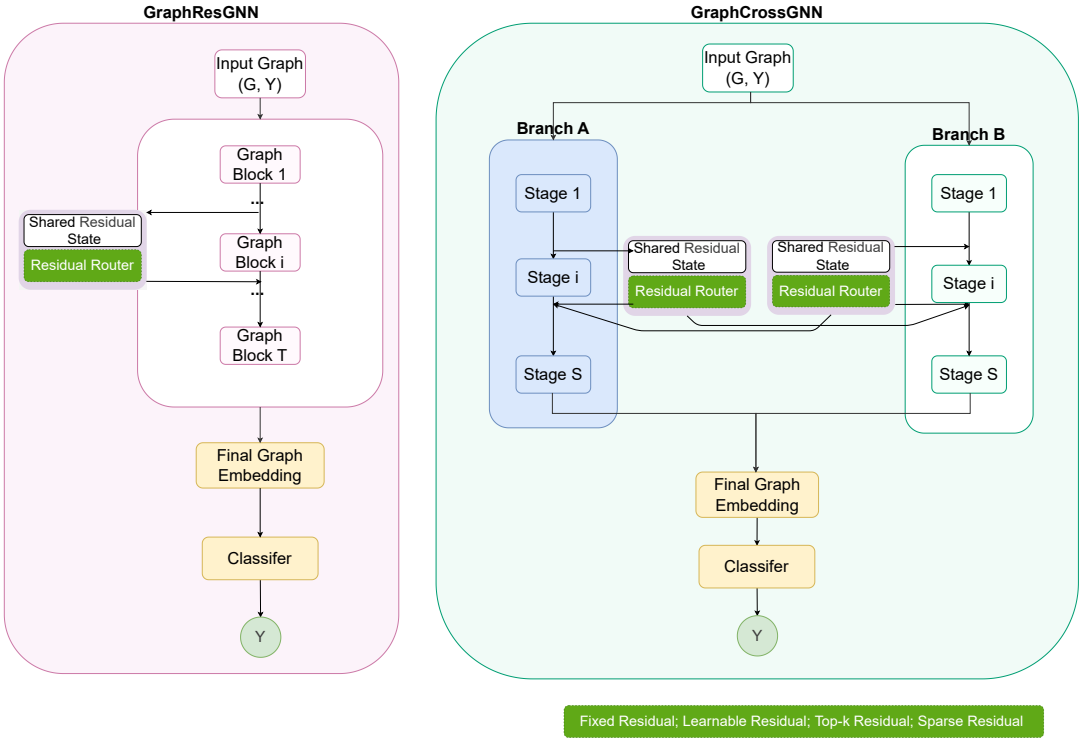

Figure 2. Graph-level CR-GNN architecture family. Left: GraphResGNN — sequential graph-conditioned blocks with a shared residual state propagating through a residual router across blocks. Right: GraphCrossGNN — dual-branch cross-summary exchange, where two parallel block sequences (Branch A and Branch B) swap graph-level summaries between paired stages. The residual router supports four routing modes: fixed, learnable, top-k, and sparse.

2.4 Residual routing mechanisms

Beyond the architecture family itself, this study evaluates how residual information is injected. This second layer is necessary because the architecture comparison alone cannot answer whether an apparent residual advantage comes from the existence of an extra path or from the way that path is filtered. Three routing modes are used in the targeted ablations:

- **Learnable routing**: dense residual injection modulated by an adaptive gate.
- **Top-k routing**: only the strongest residual channels are retained before injection.
- **Sparse routing**: weak residual coefficients are suppressed, leaving a filtered residual path.

These routing modes are not used to redefine the core architecture family. Instead, they form a mechanism layer on top of selected residual and cross-residual targets in the experimental design. The routing study is therefore interpretive rather than decorative: it tests whether the same architecture behaves differently when residual information is passed forward densely, selectively, or sparsely.

2.5 Datasets

To test the proposed architecture in a biologically meaningful setting, the benchmarks are organized into two tiers.

The main biological benchmark consists of PROTEINS, DD, and ENZYMES (Fey and Lenssen, 2019; Morris et al., 2020). The supplementary robustness suite consists of MUTAG, AIDS, and Mutagenicity. This organization keeps the main biological claim centered on protein-oriented datasets while still testing structural robustness on additional molecular graphs.

PROTEINS PROTEINS contains graph representations of proteins, where nodes denote secondary structure elements and edges encode neighborhood relations between them. The task is binary graph classification. The dataset contains 1,113 graphs, 2 classes, and 3 input features per node.

DD DD is a protein structure dataset in which each graph corresponds to a protein and nodes represent amino acids or structure-related units linked by proximity-derived edges. The task is binary classification. DD contains 1,178 graphs, 2 classes, and 89 input features.

ENZYMES ENZYMES is a 6-class enzyme classification benchmark with 600 graphs and 3 input features. It preserves an enzyme-oriented biological interpretation while remaining directly comparable to the other graph classification benchmarks in the study.

All datasets are accessed through a unified benchmark interface (Fey and Lenssen, 2019; Morris et al., 2020). We keep preprocessing light and do not introduce task-specific feature engineering, graph augmentation, or edge redesign.

All reported results follow the same stratified two-stage evaluation design:

- **Outer evaluation**: stratified 5-fold cross-validation at the graph level
- **Inner validation**: a stratified validation subset sampled from the training fold with validation ratio 0.1

The primary evaluation metric is graph classification accuracy on the held-out test fold. For fold k with test set $\mathscr{T}_k$, the fold accuracy is

$$\mathrm{Acc}_k = \frac{1}{|\mathscr{T}_k|} \sum_{(\mathscr{G}_i, y_i) \in \mathscr{T}_k} \mathbf{1}[\hat{y}_i = y_i]\,. \tag{28}$$

Across the five folds, we report the mean accuracy

$$\overline{\mathrm{Acc}} = \frac{1}{5} \sum_{k=1}^{5} \mathrm{Acc}_k \tag{29}$$

and the corresponding standard deviation

$$\sigma_{\mathrm{Acc}} = \sqrt{\frac{1}{5} \sum_{k=1}^{5} \left(\mathrm{Acc}_k - \overline{\mathrm{Acc}}\right)^2}. \tag{30}$$

The benchmark evaluates five architectures under four message-passing operators (GCNConv, GATConv, SAGEConv, GINConv).

2.6 Training configuration and implementation details

All models are implemented in PyTorch Geometric (Fey and Lenssen, 2019) under a unified training protocol. The key hyperparameters are summarized in Table 3. Training is terminated by early stopping when validation loss fails to improve for 60 consecutive epochs, and the checkpoint with the lowest validation loss is retained for test evaluation. No batch normalization or layer normalization is applied; the only regularization is dropout and weight decay. All train–validation–test splits are fixed by a global random seed of 1024, ensuring identical data partitions across models within each fold. No task-specific feature engineering, graph augmentation, or edge redesign is applied; the raw node features and adjacency structures from TUDataset are used directly.

Table 1. Unified summary of the main biological benchmark package.

Dataset	Split	#Graphs	#Cls.	Feat.	Avg. N / E	Role
PROTEINS	5-fold CV + val	1113	2	3	39.1 / 72.8	main
DD	5-fold CV + val	1178	2	89	284.3 / 715.7	main
ENZYMES	5-fold CV + val	600	6	3	32.6 / 62.1	main

Table 2. Supplementary robustness datasets retained outside the core biological narrative.

Dataset	Split	#Graphs	#Cls.	Feat.	Avg. N / E	Role
MUTAG	5-fold CV + val	188	2	7	17.9 / 19.8	supp.
AIDS	5-fold CV + val	2000	2	38	15.7 / 16.2	supp.
Mutagenicity	5-fold CV + val	4337	2	14	30.0 / 30.6	supp.

The learnable gate used in residual and cross-residual variants applies a sigmoid activation to a scalar parameter initialized at 0.8, allowing each residual injection to be adaptively scaled during training. For the top-k routing mode, only the strongest fraction of residual channels (determined by absolute magnitude) is retained before injection. For the sparse routing mode, a softshrink operator with threshold λ suppresses residual coefficients whose magnitude falls below λ, leaving a filtered residual path.

2.7 Experimental design

The empirical study separates confirmatory five-fold evidence from exploratory analyses. The benchmark evaluates six datasets, five architectures, four message-passing operators, and five outer folds under a unified protocol. The benchmark is divided into a biological core (PROTEINS, DD, ENZYMES) and a supplementary robustness suite (MUTAG, AIDS, Mutagenicity).

To answer the research questions posed in the Introduction, the study is organized into four experimental layers:

1. **Full benchmark.** All five architectures are compared across six datasets and four operators under stratified five-fold evaluation, establishing the performance hierarchy and identifying dataset–operator combinations where each topology excels.

2. **Depth-scaling analysis.** Hidden-state similarity (CKA and cosine) and per-layer gradient norms are tracked across hidden layers on PROTEINS under GCNConv, probing the representational dynamics that underlie the benchmark patterns.

3. **Gate and routing ablations.** On representative targets spanning both residual-oriented and cross-oriented settings, learnable versus fixed-strength gates are compared, and four routing modes (learnable dense, top-k at three retention ratios, sparse at three thresholds) are evaluated under the same five-fold protocol.

4. **Parameter sweeps and sensitivity checks.** Residual-parameter sweeps vary top-k ratios and sparsity strengths on the same representative targets. Single-fold sensitivity analyses over depth, dropout, and learning rate are reported as exploratory evidence only.

The five-fold results serve as confirmatory evidence throughout.

2.8 Evaluation metrics and statistical reporting

The primary metric is graph classification accuracy on the held-out test fold. Main benchmark and targeted ablation results are reported as five-fold mean accuracy with standard deviation. Five-fold experiments provide confirmatory evidence; single-fold parameter scans are reported as exploratory analyses only.

3 RESULTS

All results are reported as five-fold mean best test accuracy $\pm$ fold-level standard deviation; single-fold sensitivity analyses are exploratory only.

Table 3. Key training hyperparameters shared across all experiments.

Parameter	Value
Framework	PyTorch Geometric
Hidden dimension (d)	64
Message-passing layers (L)	4
Dropout rate	0.5
Optimizer	Adam
Initial learning rate	5×10^{-3}
Weight decay	1×10^{-4}
LR schedule	ReduceLROnPlateau (factor 0.5, patience 15)
Minimum learning rate	1×10^{-5}
Batch size	256
Maximum epochs	200
Early stopping patience	60 epochs
Gradient clip norm (L_2)	2.0
Pooling	Global mean pooling
Loss function	Cross-entropy
Random seed	1024
Validation split ratio	0.1

3.1 Overall benchmark performance

The benchmark evaluates six datasets and five CR-GNN architectures under GCNConv with five outer folds. For the paper-facing argument, Table 4 is the main result table: it contains the protein-oriented comparison centered on PROTEINS and DD, plus ENZYMES as a supporting biological stress test, whereas Table 5 is retained as supplementary robustness evidence. Within the main pair, NodeResGNN is best on PROTEINS (0.7143) and GraphResGNN is best on DD (0.7275). The margins are often small: on PROTEINS, GraphResGNN trails NodeResGNN by only 0.0072, and on DD the three strongest models (GraphResGNN, GraphCrossGNN, NodeCrossGNN) all lie within 0.0111 accuracy points.

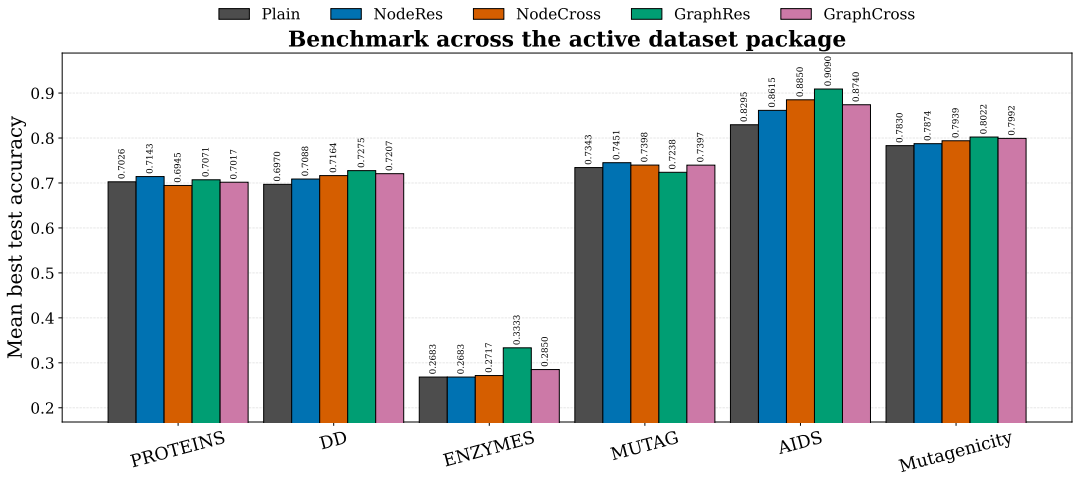

Figure 3. Full V3 benchmark across the active dataset package under the GCNConv comparison. Bars show five-fold mean best test accuracy with standard-deviation error bars.

Figure 3 makes the same pattern visually explicit. Across the full suite, graph-level residual propagation is the strongest default choice, but the main biological claim should be read from the protein-oriented core rather than from the full six-dataset average impression.

Table 4. Main biological benchmark results. Cells show mean ± std [min, max] over five folds. Bold: best mean per dataset.

Model	PROTEINS	DD	ENZYMES
PlainGNN	0.7026 ± 0.0331 [0.655, 0.735]	0.6970 ± 0.0421 [0.646, 0.763]	0.2683 ± 0.0509 [0.217, 0.358]
NodeResGNN	**0.7143 ± 0.0223 [0.685, 0.744]**	0.7088 ± 0.0332 [0.651, 0.754]	0.2683 ± 0.0517 [0.217, 0.367]
NodeCrossGNN	0.6945 ± 0.0488 [0.613, 0.735]	0.7164 ± 0.0315 [0.677, 0.771]	0.2717 ± 0.0692 [0.167, 0.367]
GraphResGNN	0.7071 ± 0.0402 [0.655, 0.749]	**0.7275 ± 0.0144 [0.711, 0.747]**	**0.3333 ± 0.1000 [0.150, 0.425]**
GraphCrossGNN	0.7017 ± 0.0520 [0.608, 0.757]	0.7207 ± 0.0322 [0.694, 0.784]	0.2850 ± 0.0868 [0.167, 0.408]

Table 5. Supplementary robustness benchmark results. Cells show mean ± std [min, max] over five folds. Bold: best mean per dataset.

Model	MUTAG	AIDS	Mutagenicity
PlainGNN	0.7343 ± 0.0433 [0.684, 0.811]	0.8295 ± 0.0456 [0.755, 0.877]	0.7830 ± 0.0170 [0.764, 0.812]
NodeResGNN	**0.7451 ± 0.0504 [0.684, 0.838]**	0.8615 ± 0.0314 [0.800, 0.885]	0.7874 ± 0.0142 [0.769, 0.809]
NodeCrossGNN	0.7398 ± 0.0523 [0.684, 0.838]	0.8850 ± 0.0380 [0.818, 0.930]	0.7939 ± 0.0196 [0.770, 0.826]
GraphResGNN	0.7238 ± 0.0778 [0.649, 0.865]	**0.9090 ± 0.0194 [0.890, 0.935]**	**0.8022 ± 0.0244 [0.763, 0.832]**
GraphCrossGNN	0.7397 ± 0.0531 [0.684, 0.838]	0.8740 ± 0.0460 [0.800, 0.943]	0.7992 ± 0.0232 [0.756, 0.826]

3.2 Biological core and cross-residual advantage

The biological core is the most important part of the paper-facing benchmark, but within that core the main biological comparison is carried by PROTEINS and DD. ENZYMES is retained as a supporting stress test, so we examine all three while keeping that hierarchy explicit.

Figure 4 shows that the core datasets do not support a single biological narrative. On PROTEINS, NodeResGNN is best and GraphResGNN is second. On DD, the ordering shifts: GraphResGNN is best in the GCNConv table, but the two cross-residual models remain close behind. These two datasets carry the main biological comparison, and together they support a split regime rather than a universal winner. On ENZYMES, GraphResGNN is again best, but all models perform poorly, with the best mean accuracy only 0.3333 on a 6-class task. ENZYMES therefore serves as a supporting difficulty stress test rather than as a primary separator between reuse topologies.

3.3 Supplementary robustness reference

The supplementary robustness datasets are summarized in Appendix A. Their ordering remains mixed—NodeResGNN is best on MUTAG, whereas GraphResGNN is best on AIDS and Mutagenicity—which is useful as a robustness check, but these datasets do not define the central biological claim.

Figure 5 clarifies what can and cannot be claimed for cross-residual exchange across the full suite. The best cross-residual variant outperforms PlainGNN on five of six datasets, with the largest gains on ENZYMES (+0.057), AIDS (+0.028), and Mutagenicity (+0.022). However, the best cross-residual variant surpasses the best residual variant only on MUTAG (+0.005) and Mutagenicity (+0.001), while essentially tying on DD (-0.001). Cross-residual exchange is therefore a useful option, but not a universal replacement for the strongest residual baseline, including within the protein-oriented core.

3.4 The mechanism: representational continuity vs. performance

Why does GraphResGNN—which deliberately alters representations between blocks—outperform architectures that preserve them more faithfully? To answer this, we track hidden-state similarity (CKA and cosine) and per-layer gradient norms from $L = 1$ to $L = 8$ on PROTEINS under GCNConv.

Figure 6 reveals a counter-intuitive pattern. At $L = 8$, the CKA ordering is NodeResGNN (0.968) > GraphCrossGNN (0.952) > PlainGNN (0.951) > NodeCrossGNN (0.946) > GraphResGNN (0.942). The architecture that preserves representations most faithfully (NodeResGNN) wins only 3 of 24 combinations; the architecture that reshapes them most aggressively (GraphResGNN) wins 12 of 24. We term this the **CKA–performance paradox**: stronger layer-wise representational continuity does not predict stronger final accuracy. The cosine trends follow the same ordering and are omitted from the main text for brevity.

This paradox has a structural explanation. GraphResGNN's three graph-conditioned blocks each receive the previous block's pooled graph embedding as a conditioning signal. This deliberate, structured reshaping—reinjecting accumulating global context at each stage—produces more discriminative final representations precisely *because* representations change substantially between blocks. Conversely,

Table 6. Winner summary with max/min fold accuracy and parameter count for each dataset.

Dataset	Winner	Mean Acc	Max Acc	Min Acc
PROTEINS	NodeResGNN	0.7143	0.7444	0.6847
DD	GraphResGNN	0.7275	0.7468	0.7106
ENZYMES	GraphResGNN	0.3333	0.4250	0.1500
MUTAG	NodeResGNN	0.7451	0.8378	0.6842
AIDS	GraphResGNN	0.9090	0.9350	0.8900
Mutagenicity	GraphResGNN	0.8022	0.8316	0.7629

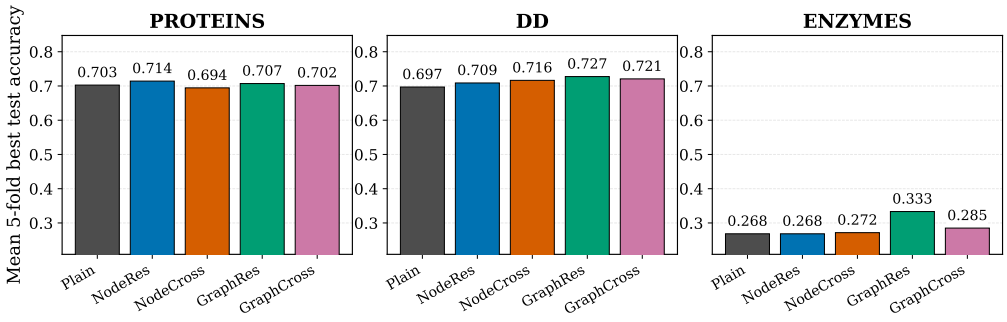

Figure 4. V3 results on the protein-oriented core. The main comparison is carried by PROTEINS and DD; ENZYMES is retained as a supporting stress test.

NodeResGNN's faithful preservation of representations across depth may limit its capacity to adapt to task-specific structure.

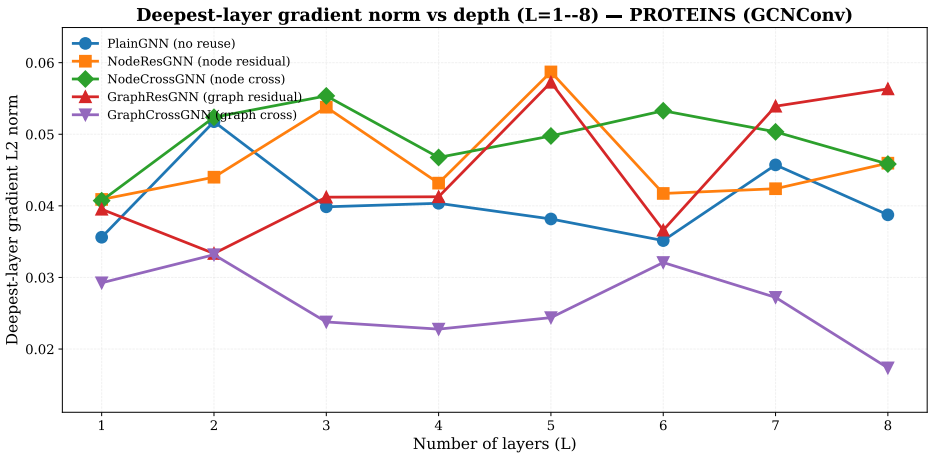

Figure 7. Deepest-layer gradient L2 norm across $L = 1$–8 (PROTEINS, GCNConv).

Figure 7 confirms that PlainGNN's deepest gradient norm degrades fastest with depth; the residual family maintains more stable gradients throughout. NodeResGNN and NodeCrossGNN exhibit comparable gradient stability, confirming that cross-branch exchange does not introduce gradient interference.

3.5 Quantifying the trade-offs

To move beyond qualitative patterns, Table 7 counts per-architecture wins across all 24 dataset–operator combinations, and Table 8 measures cross-dataset rank stability.

The trade-off is now quantitative. Across the 24 dataset–operator combinations, GraphResGNN wins most often (12/24) but with relatively weak rank stability (std 1.46)—it can win clearly or fall to the bottom, as on MUTAG. GraphCrossGNN wins only once, but ties PlainGNN for the smallest rank standard deviation (0.75) while remaining much closer to the top of the ranking. NodeCrossGNN occupies

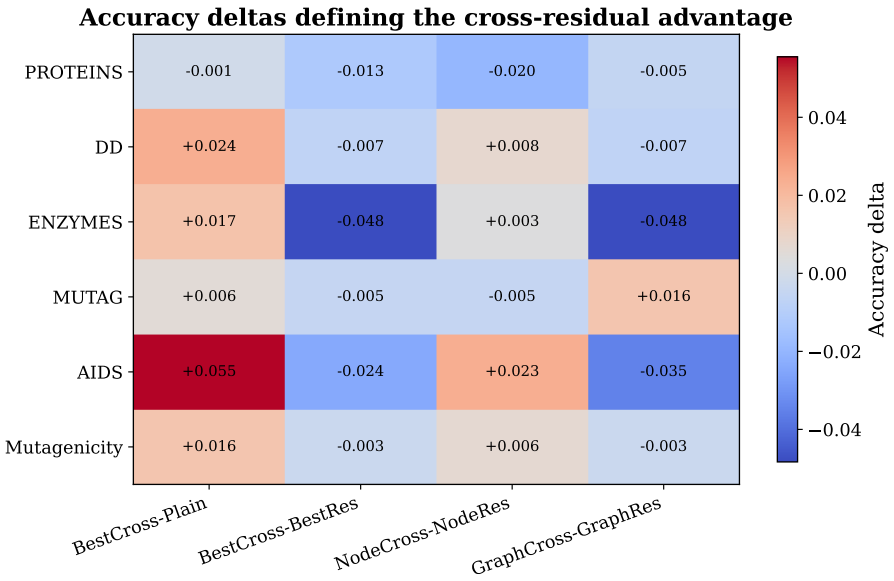

Figure 5. V3 cross-residual advantage heatmap. Positive cells indicate that the best cross-residual variant outperforms the indicated comparator on that dataset.

Table 7. Winner counts across 24 dataset–operator combinations (CR-GNN family).

Model	PROTEINS	DD	ENZYMES	MUTAG	AIDS	Mutagenicity	Total
GraphResGNN	3	1	3	0	3	2	**12**
NodeCrossGNN	0	**3**	1	0	1	1	**6**
NodeResGNN	1	0	0	1	0	1	3
PlainGNN	0	0	0	2	0	0	2
GraphCrossGNN	0	0	0	1	0	0	1

the middle: second-most wins (6/24), intermediate stability (std 1.00), and a clear concentration of wins on DD at the operator level.

3.6 Ablation evidence

3.6.1 Gate ablation

Gate ablations test whether adaptive control of residual injection explains the architectural differences observed above. On the strongest graph-level residual target (`AIDS + GraphResGNN + GINConv`), the learnable gate performs best, though fixed coefficient 0.5 is close. On six cross-favored targets, learnable and fixed gates each win three—cross-residual architectures are robust to gate configuration, consistent with their greater rank stability.

3.6.2 Residual routing

Four representative targets compare learnable dense, top-k, and sparse routing modes. Sparse routing (threshold 0.05) is optimal on three of four targets; learnable dense routing remains best only on the strongest graph-level residual target. Two of the three sparse-favoring targets involve cross-residual architectures, suggesting that sparse filtering and cross-branch exchange are complementary rather than universally dominant.

3.6.3 Statistical tests

The paired-test matrix for the main biological evidence (Table 14) confirms that most architecture-level gaps are modest relative to fold-level variation. The corresponding supplementary-dataset tests are reported in Appendix A.

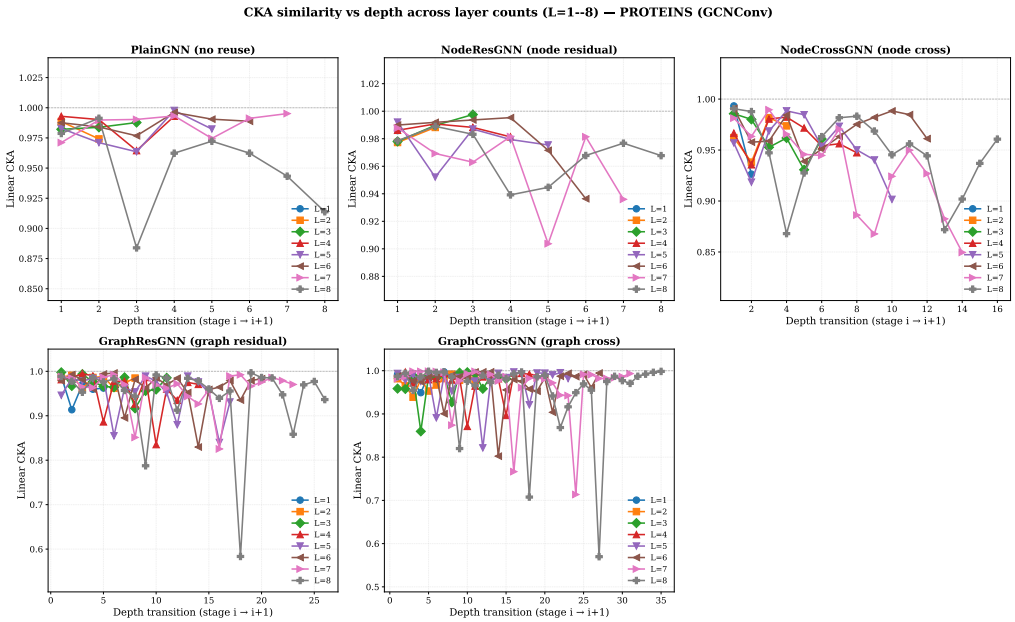

Figure 6. CKA similarity between adjacent depth stages across $L = 1$–8 on PROTEINS under GCNConv.

Table 8. Mean rank and rank stability across six datasets (GCNConv).

Model	Mean rank	Gap to best	Rank std
GraphResGNN	1.8	0.005	1.46
GraphCrossGNN	2.7	0.018	**0.75**
NodeCrossGNN	3.0	0.022	1.00
NodeResGNN	3.2	0.024	1.57
PlainGNN	4.3	0.036	**0.75**

3.6.4 Parameter sweep

Additional top-k ratios and sparsity levels refine rather than change the pattern. The strongest top-k ratio varies by target, and aggressive sparsity (0.10) damages the graph-level residual target, confirming that sparsity benefits are threshold-dependent.

3.7 Exploratory sensitivity analysis

Auxiliary sensitivity checks over depth, dropout, and learning rate on PROTEINS, DD, and ENZYMES do not materially change the conclusions above.

4 DISCUSSION

This section returns to the three research questions posed in the Introduction and answers them in light of the evidence presented in Section 3.

4.1 RQ1: Why does the architecture with the weakest representational continuity win most often?

The CKA–performance paradox—GraphResGNN achieves the lowest CKA at depth (0.942) yet the most wins (12/24), while NodeResGNN achieves the highest CKA (0.968) yet only 3 wins—has a structural explanation. GraphResGNN chains three graph-conditioned blocks, each receiving the previous block's pooled graph embedding as conditioning input. This design deliberately reshapes representations at every stage using accumulating global context: it does not aim to preserve representations, but to progressively refine them. In the present benchmark, that strategy is strongest on DD, ENZYMES, AIDS, and Mutagenicity, but it is not uniformly best.

Table 9. AIDS gate ablation on `GraphResGNN + GINConv`. Numbers are five-fold mean accuracy ± standard deviation. Bold: best per column.

Gate setting	Mean accuracy
Learnable	**0.9620 ± 0.0089**
Fixed 0.0	0.9150 ± 0.0138
Fixed 0.5	0.9605 ± 0.0144
Fixed 1.0	0.9480 ± 0.0300

Table 10. Fold-level AIDS gate ablation on `AIDS + GraphResGNN + GINConv`. Bold: best per column.

Gate setting	Fold 0	Fold 1	Fold 2	Fold 3	Fold 4	Mean ± Std
Learnable	0.9500	0.9700	0.9700	0.9675	0.9525	**0.9620 ± 0.0089**
Fixed 0.0	0.9225	0.9025	0.9125	0.9000	0.9375	0.9150 ± 0.0138
Fixed 0.5	0.9325	0.9625	0.9675	0.9675	0.9725	0.9605 ± 0.0144
Fixed 1.0	0.8925	0.9450	0.9700	0.9775	0.9550	0.9480 ± 0.0300

NodeResGNN preserves representations faithfully but at a cost: representations that change too little between layers may fail to adapt to task-specific structure. The cross-residual architectures partly resolve this tension by providing an alternative pathway for information a single branch might suppress, without forcing the aggressive reshaping that can make GraphResGNN brittle. This is why GraphCrossGNN remains consistently near the top of the ranking even though it rarely finishes first.

Answer: Within this study, aggressive representational reshaping can be productive, but only in a dataset-dependent way. Higher continuity is not sufficient for better accuracy, and cross-residual exchange is best understood as a stabilizing alternative rather than proof that continuity should always be reduced.

4.2 RQ1 & RQ2: When does each reuse topology dominate?

The benchmark supports a split regime rather than a single winner. In the paper-facing biological comparison, NodeResGNN is strongest on PROTEINS, while GraphResGNN is strongest on DD. Cross-residual variants are most convincing on DD: in the operator-level winner count, NodeCrossGNN wins three of four DD operator settings even though GraphResGNN remains the best DD model in the single-operator GCNConv table. This distinction matters because it shows that the DD story is robust for cross-residual exchange across operators, but not reducible to a single-model sweep. ENZYMES points in the same GraphResGNN direction, but because all models perform poorly there, it is better interpreted as a stress test than as co-equal main evidence.

On datasets where the graph-level summary appears more useful, GraphResGNN is the stronger default choice. On datasets where the best residual baseline already matches or exceeds the best cross-residual variant, cross exchange should be interpreted as a competitive alternative rather than the preferred architecture by default. Figure 5 makes this boundary explicit across the full suite: the best cross variant usually improves on PlainGNN, but only rarely improves on the best residual model. The supplementary datasets are useful for checking that this boundary is not unique to the protein-oriented core, but they should not redefine the main biological argument.

Answer: Choose graph-level residual propagation as the strongest default family in the current benchmark, choose node-level residual reuse when the task resembles PROTEINS, and treat cross-residual exchange as a selective option that is especially worth testing on DD-like protein graphs and in operator-diverse comparisons.

4.3 RQ3: Does routing strategy matter beyond architecture choice?

The ablation results (Section 3) show that it does, and in a target-dependent way. First, gate ablations reveal that cross-residual architectures are robust to gate configuration—learnable and fixed gates split evenly on cross-favored targets—consistent with their stable ranking profile. Second, routing mode matters substantially: sparse routing (threshold 0.05) is optimal on three of four representative targets, and two of these involve cross-residual architectures. The mechanism-level synergy is that sparsity suppresses weak residual channels before they accumulate as noise across depth, while the cross-branch pathway preserves genuinely complementary signals through an independent route. Aggressive sparsity (0.10)

Table 11. Cross-gate ablation on six cross-favored targets. Numbers are five-fold mean accuracy ± standard deviation. Bold: best per row.

Dataset	Target	Learnable	Fixed 0.0	Fixed 0.5	Fixed 1.0
AIDS	NodeCross + GATConv	0.9045 ± 0.0165	0.9075 ± 0.0065	**0.9195 ± 0.0114**	0.9105 ± 0.0241
DD	NodeCross + GATConv	**0.7173 ± 0.0271**	0.7053 ± 0.0361	0.7156 ± 0.0225	0.7147 ± 0.0288
DD	NodeCross + GINConv	0.7071 ± 0.0306	0.7012 ± 0.0198	**0.7232 ± 0.0207**	0.7113 ± 0.0277
ENZYMES	NodeCross + GATConv	0.2800 ± 0.0629	**0.2850 ± 0.0627**	0.2550 ± 0.0674	0.2650 ± 0.0690
MUTAG	GraphCross + GATConv	**0.7558 ± 0.0495**	0.7451 ± 0.0407	0.7450 ± 0.0416	0.7504 ± 0.0779
Mutagenicity	NodeCross + GATConv	**0.8114 ± 0.0110**	0.8033 ± 0.0192	0.8010 ± 0.0130	0.8036 ± 0.0078

Table 12. Residual-routing and parameter-sweep summary on four representative targets (transposed). Numbers are five-fold mean accuracy ± standard deviation. Bold: best per column.

Routing	AIDS+GraphRes+GIN	PROT+GraphRes+SAGE	AIDS+NodeCross+GAT	DD+NodeCross+GAT
Learnable	**0.9500 ± 0.0087**	0.7178 ± 0.0411	0.8970 ± 0.0491	0.7080 ± 0.0172
Top-k 0.25	0.9075 ± 0.0237	0.7241 ± 0.0407	0.9160 ± 0.0128	0.7062 ± 0.0249
Top-k 0.50	0.9330 ± 0.0251	0.7151 ± 0.0397	0.8960 ± 0.0483	0.7029 ± 0.0160
Top-k 0.75	0.9240 ± 0.0287	0.7214 ± 0.0318	0.8960 ± 0.0489	0.7164 ± 0.0248
Sparse 0.02	0.9470 ± 0.0181	0.7160 ± 0.0248	0.9060 ± 0.0268	0.7147 ± 0.0259
Sparse 0.05	0.9250 ± 0.0152	**0.7259 ± 0.0475**	**0.9180 ± 0.0162**	**0.7181 ± 0.0196**
Sparse 0.10	0.8315 ± 0.0549	0.7088 ± 0.0521	0.9025 ± 0.0139	0.7105 ± 0.0243
Best	Learnable	Sparse 0.05	Sparse 0.05	Sparse 0.05

damages the graph-level residual target, confirming that the effect is threshold-dependent rather than universal.

Answer: Routing strategy is a meaningful secondary design axis. Mild sparse routing is often the best companion to cross-residual exchange, whereas learnable dense routing remains strongest on the best graph-level residual target.

4.4 Limitations

Several limitations bound the generality of these findings. First, statistical power is limited: with five folds, only large effects reach $p < 0.05$. Second, the CKA–performance paradox was documented on a single dataset–operator combination (PROTEINS, GCNConv); whether it generalizes remains open. Third, the benchmark datasets are medium-scale biomolecular graphs; the trade-offs may shift on larger or different graph types. Fourth, the routing analysis covers four targets—sufficient to reveal the pattern, but not to claim universality.

4.5 Future directions

The most important extension is to test the CKA–performance paradox on larger benchmarks with richer inputs. If it generalizes, it would reframe how deep GNN architectures are evaluated—shifting focus from layer-wise stability metrics to the structuredness of representational change. The sparse-plus-cross-residual synergy should also be tested beyond the four targets studied here.

5 CONCLUSIONS

This paper compares five residual routing topologies under a controlled graph-classification protocol and finds that peak accuracy, cross-dataset consistency, and representational continuity are systematically traded off. The main contribution is the controlled comparison framework that makes these trade-offs quantitatively visible without changing the backbone, pooling rule, or evaluation protocol across models.

Returning to the three research questions: (1) the strongest architecture depends on the dataset regime rather than on a universal residual rule; in the main protein-oriented comparison, NodeResGNN is best on PROTEINS whereas GraphResGNN is best on DD. ENZYMES points in the same GraphResGNN direction, but mainly as a supporting stress test rather than as co-equal main evidence. (2) Cross-residual exchange is useful but selective: it usually improves on PlainGNN, remains competitive on DD and Mutagenicity, and becomes especially relevant in the operator-level DD comparison, but it does not consistently outperform the best residual baseline. (3) Routing strategy is a meaningful secondary design axis: mild sparse routing is most often optimal in the targeted ablations, while learnable dense routing remains strongest on the best graph-level residual target.

Table 13. Family-wise routing winners.

Target	Best learnable	Best top-k	Best sparse	Overall winner
AIDS + GraphRes + GIN	0.9500	0.9330	0.9470	Learnable
PROTEINS + GraphRes + SAGE	0.7178	0.7241	0.7259	Sparse
AIDS + NodeCross + GAT	0.8970	0.9160	0.9180	Sparse
DD + NodeCross + GAT	0.7080	0.7164	0.7181	Sparse

Table 14. Paired t-tests for main biological benchmark datasets. Cells: Δ (Cohen's d, p). **Bold**: $p < 0.05$ (n=5 folds).

Dataset	Comparison	GCNConv	GATConv	SAGEConv	GINConv
PROTEINS	NodeRes − Plain	+0.012 (+0.55, 0.286)	+0.000 (+0.00, 0.999)	+0.006 (+0.31, 0.524)	+0.012 (+0.56, 0.277)
	NodeCross − Plain	-0.008 (-0.28, 0.571)	-0.011 (-0.22, 0.647)	-0.002 (-0.07, 0.881)	-0.008 (-0.23, 0.632)
	GraphRes − Plain	+0.004 (+0.45, 0.375)	+0.015 (+0.48, 0.345)	**+0.030 (+1.69, 0.019)**	+0.040 (+1.10, 0.069)
	GraphCross − Plain	-0.001 (-0.02, 0.961)	-0.008 (-0.24, 0.623)	+0.004 (+0.24, 0.622)	+0.004 (+0.16, 0.743)
	NodeCross − NodeRes	-0.020 (-0.53, 0.302)	-0.011 (-0.19, 0.690)	-0.008 (-0.72, 0.181)	-0.020 (-0.87, 0.125)
	GraphCross − GraphRes	-0.005 (-0.15, 0.751)	**-0.023 (-2.18, 0.008)**	**-0.026 (-1.47, 0.031)**	**-0.036 (-2.36, 0.006)**
	GraphRes − NodeRes	-0.007 (-0.26, 0.588)	+0.015 (+0.37, 0.459)	+0.023 (+0.75, 0.171)	+0.028 (+1.05, 0.079)
	NodeCross − GraphCross	-0.007 (-0.49, 0.337)	-0.003 (-0.10, 0.838)	-0.005 (-0.21, 0.659)	-0.012 (-0.39, 0.427)
DD	NodeRes − Plain	+0.012 (+0.39, 0.429)	+0.002 (+0.24, 0.624)	+0.016 (+0.48, 0.347)	+0.001 (+0.03, 0.946)
	NodeCross − Plain	+0.019 (+0.55, 0.286)	+0.035 (+0.90, 0.115)	+0.023 (+0.87, 0.124)	+0.025 (+0.93, 0.107)
	GraphRes − Plain	+0.030 (+0.64, 0.223)	+0.020 (+0.50, 0.326)	+0.021 (+0.91, 0.112)	+0.002 (+0.06, 0.892)
	GraphCross − Plain	+0.024 (+0.87, 0.123)	+0.019 (+0.46, 0.357)	+0.004 (+0.19, 0.699)	+0.008 (+0.20, 0.672)
	NodeCross − NodeRes	+0.008 (+0.43, 0.395)	+0.033 (+0.88, 0.121)	+0.007 (+0.64, 0.226)	+0.024 (+0.94, 0.103)
	GraphCross − GraphRes	-0.007 (-0.23, 0.637)	-0.002 (-0.11, 0.815)	**-0.017 (-2.82, 0.003)**	+0.006 (+0.14, 0.762)
	GraphRes − NodeRes	+0.019 (+0.64, 0.228)	+0.019 (+0.46, 0.359)	+0.005 (+0.24, 0.624)	+0.001 (+0.03, 0.954)
	NodeCross − GraphCross	-0.004 (-0.31, 0.526)	+0.016 (+1.07, 0.075)	**+0.019 (+1.53, 0.027)**	+0.017 (+0.57, 0.274)
ENZYMES	NodeRes − Plain	-0.000 (-0.00, 1.000)	+0.005 (+0.19, 0.697)	-0.017 (-0.94, 0.103)	+0.030 (+0.47, 0.351)
	NodeCross − Plain	+0.003 (+0.09, 0.847)	+0.032 (+0.45, 0.369)	+0.007 (+0.24, 0.614)	**+0.032 (+2.12, 0.009)**
	GraphRes − Plain	+0.020 (+0.92, 0.107)	+0.020 (+0.32, 0.507)	+0.020 (+0.47, 0.360)	**+0.068 (+2.45, 0.002)**
	GraphCross − Plain	+0.020 (+0.55, 0.280)	+0.007 (+0.08, 0.875)	+0.020 (+0.33, 0.502)	+0.028 (+0.52, 0.303)
	NodeCross − NodeRes	+0.003 (+0.08, 0.878)	+0.027 (+0.36, 0.471)	+0.023 (+0.42, 0.399)	+0.002 (+0.07, 0.884)
	GraphCross − GraphRes	-0.000 (-0.01, 0.981)	-0.013 (-0.21, 0.672)	+0.000 (+0.00, 1.000)	-0.040 (-0.62, 0.237)
	GraphRes − NodeRes	+0.020 (+0.57, 0.264)	+0.015 (+0.28, 0.562)	+0.037 (+0.82, 0.151)	+0.038 (+1.02, 0.079)
	NodeCross − GraphCross	-0.017 (-0.47, 0.355)	+0.025 (+0.68, 0.211)	-0.013 (-0.40, 0.444)	+0.003 (+0.07, 0.894)

The depth-scaling analysis on PROTEINS (GCNConv) reveals a "CKA–performance paradox" that warrants broader validation: the architecture with the highest representational continuity (NodeResGNN) does not win most often, while the architecture with the lowest continuity (GraphResGNN) wins most often in the broader dataset–operator sweep. This pattern suggests—but does not yet prove—that structured representational change between layers can be productive rather than destructive.

The strongest claims are supported by the stratified five-fold benchmark on the protein-oriented core, the targeted five-fold ablations, and the $L = 1$–8 depth-scaling analysis. The study addresses controlled graph-classification behavior on medium-scale biomolecular benchmarks; future work should test whether the CKA–performance paradox and the sparse-plus-cross-residual synergy generalize to larger graphs, richer biological inputs, and datasets that move beyond benchmark-level biomolecular classification.

6 ACKNOWLEDGMENTS

The authors thank the open-source graph learning community and the maintainers of the benchmark resources used in this study. They also thank the reviewers and editors for their time and feedback.

A APPENDIX

A.1 Supplementary robustness benchmark

The datasets in this section are retained as robustness checks outside the main protein-oriented comparison. They are informative about how the architecture ordering behaves away from the PROTEINS–DD pair, but they do not define the main biological claim of the paper.

A.2 Supplementary statistical tests

Table 15. Main biological benchmark results. Numbers are mean best test accuracy ± standard deviation.

Model	PROTEINS	DD	ENZYMES
PlainGNN	0.7053 ± 0.0271	0.7165 ± 0.0179	0.2450 ± 0.0407
NodeResGNN	0.6945 ± 0.0509	$\mathbf{0.7207 \pm 0.0259}$	0.2650 ± 0.0661
NodeCrossGNN	0.6963 ± 0.0433	0.7198 ± 0.0178	0.3000 ± 0.0580
GraphResGNN	$\mathbf{0.7223 \pm 0.0367}$	0.6934 ± 0.0458	$\mathbf{0.3117 \pm 0.0746}$
GraphCrossGNN	0.6864 ± 0.0465	0.7097 ± 0.0309	0.3017 ± 0.0750

Table 16. Supplementary robustness benchmark results.

Model	MUTAG	AIDS	Mutagenicity
PlainGNN	0.7236 ± 0.0486	0.8455 ± 0.0257	0.7814 ± 0.0235
NodeResGNN	0.7343 ± 0.0734	0.8690 ± 0.0354	0.7897 ± 0.0139
NodeCrossGNN	$\mathbf{0.7397 \pm 0.0676}$	0.8735 ± 0.0412	$\mathbf{0.8033 \pm 0.0159}$
GraphResGNN	0.7290 ± 0.0585	$\mathbf{0.9160 \pm 0.0121}$	0.8022 ± 0.0220
GraphCrossGNN	0.7129 ± 0.0539	0.8710 ± 0.0412	0.8029 ± 0.0182

REFERENCES

Uri Alon and Eran Yahav. On the bottleneck of graph neural networks. In *International Conference on Learning Representations (ICLR)*, 2021.

Xavier Bresson and Thomas Laurent. Residual gated graph convnets. *arXiv preprint arXiv:1711.07553*, 2017.

Tianle Cai, Jiacheng Luo, Sheng Wang, Kai Zhang, Yu Tian, Liwei Yang, Zekun Li, and Kilian Weinberger. Graphnorm: A principled approach to accelerating graph neural network training. In *International Conference on Machine Learning*, pages 1277–1287, 2021.

Ming Chen, Zhewei Wei, Zengfeng Huang, Bolin Ding, and Yaliang Li. Simple and deep graph convolutional networks. In *International Conference on Machine Learning (ICML)*, pages 1725–1735, 2020a.

Yuning Chen, Xihao Wei, Pan Xu, Xinwen Wang, Ping Luo, Zhiyuan Li, and Jian Tang. Revisiting over-smoothing in deep gcns. *arXiv preprint arXiv:2003.13663*, 2020b.

Matthias Fey and Jan Eric Lenssen. Fast graph representation learning with pytorch geometric. *arXiv preprint arXiv:1903.02428*, 2019.

Hongyang Gao and Shuiwang Ji. Graph u-net. *arXiv preprint arXiv:1905.05178*, 2019.

Justin Gilmer, Samuel S Schoenholz, Patrick F Riley, Oriol Vinyals, and George E Dahl. Neural message passing for quantum chemistry. In *International Conference on Machine Learning*, pages 1263–1272, 2017.

William L Hamilton, Rex Ying, and Jure Leskovec. Inductive representation learning on large graphs. In *Advances in neural information processing systems*, pages 1024–1034, 2017.

Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 770–778, 2016.

Thomas N Kipf and Max Welling. Semi-supervised classification with graph convolutional networks. *arXiv preprint arXiv:1609.02907*, 2016.

Johannes Klicpera, Alan Bojchevski, and Stephan Günnemann. Combining label propagation and simple models out-performs graph neural networks. *arXiv preprint arXiv:1810.05997*, 2018.

Table 17. Winner summary, optimization depth, and parameter count for each dataset.

Dataset	Winner	Mean Acc	Mean Best Epoch	Params
PROTEINS	GraphResGNN	0.7223	58.8	51208
DD	NodeResGNN	0.7207	25.4	22530
ENZYMES	GraphResGNN	0.3117	91.4	52248
MUTAG	NodeCrossGNN	0.7397	88.2	34434
AIDS	GraphResGNN	0.9160	83.8	57928
Mutagenicity	NodeCrossGNN	0.8033	131.2	35330

Table 18. Paired t-tests for supplementary robustness datasets. Cells: Δ (Cohen's d, p). **Bold**: $p < 0.05$ (n=5 folds).

Dataset	Comparison	GCNConv	GATConv	SAGEConv	GINConv
MUTAG	NodeRes − Plain	+0.011 (+0.29, 0.545)	+0.000 (+0.00, 1.000)	-0.016 (-0.40, 0.416)	-0.005 (-0.12, 0.812)
	NodeCross − Plain	+0.016 (+0.36, 0.477)	+0.005 (+0.09, 0.850)	+0.005 (+0.15, 0.758)	+0.016 (+0.34, 0.498)
	GraphRes − Plain	+0.005 (+0.09, 0.864)	+0.011 (+0.33, 0.513)	-0.011 (-0.10, 0.839)	-0.016 (-0.22, 0.670)
	GraphCross − Plain	-0.011 (-0.20, 0.683)	+0.027 (+0.59, 0.262)	-0.022 (-0.28, 0.576)	-0.005 (-0.16, 0.756)
	NodeCross − NodeRes	+0.005 (+0.10, 0.836)	+0.005 (+0.10, 0.845)	+0.022 (+0.33, 0.516)	+0.022 (+0.37, 0.464)
	GraphCross − GraphRes	-0.016 (-0.34, 0.515)	+0.016 (+0.37, 0.467)	-0.011 (-0.26, 0.613)	+0.011 (+0.25, 0.618)
	GraphRes − NodeRes	-0.005 (-0.12, 0.808)	+0.011 (+0.17, 0.742)	+0.005 (+0.08, 0.881)	-0.011 (-0.12, 0.824)
	NodeCross − GraphCross	+0.027 (+0.71, 0.186)	-0.022 (-0.61, 0.240)	+0.027 (+0.53, 0.310)	+0.022 (+0.48, 0.349)
AIDS	NodeRes − Plain	+0.023 (+1.19, 0.062)	+0.010 (+0.89, 0.112)	+0.018 (+0.86, 0.125)	+0.015 (+0.59, 0.261)
	NodeCross − Plain	+0.028 (+1.06, 0.083)	+0.018 (+0.80, 0.144)	+0.020 (+1.52, 0.022)	+0.018 (+0.67, 0.222)
	GraphRes − Plain	**+0.070 (+2.41, 0.005)**	+0.025 (+1.57, 0.041)	+0.028 (+2.45, 0.004)	**+0.048 (+2.47, 0.004)**
	GraphCross − Plain	+0.025 (+0.94, 0.089)	+0.013 (+1.10, 0.068)	+0.018 (+1.16, 0.067)	+0.023 (+0.86, 0.124)
	NodeCross − NodeRes	+0.005 (+0.41, 0.420)	+0.008 (+0.52, 0.311)	+0.003 (+0.14, 0.778)	+0.003 (+1.61, 0.005)
	GraphCross − GraphRes	-0.045 (-1.46, 0.043)	-0.013 (-0.66, 0.218)	-0.010 (-0.44, 0.398)	-0.025 (-0.87, 0.125)
	GraphRes − NodeRes	**+0.047 (+2.62, 0.002)**	+0.015 (+1.19, 0.063)	+0.010 (+0.46, 0.370)	+0.033 (+2.02, 0.014)
	NodeCross − GraphCross	-0.003 (-0.10, 0.848)	+0.005 (+0.44, 0.387)	+0.003 (+0.20, 0.682)	-0.005 (-0.16, 0.751)
Mutagenicity	NodeRes − Plain	+0.004 (+0.26, 0.597)	+0.011 (+0.95, 0.101)	+0.014 (+0.93, 0.106)	+0.013 (+0.83, 0.137)
	NodeCross − Plain	+0.011 (+0.68, 0.201)	+0.020 (+1.05, 0.079)	+0.013 (+0.86, 0.127)	+0.017 (+0.95, 0.101)
	GraphRes − Plain	+0.019 (+1.24, 0.050)	+0.008 (+0.57, 0.273)	+0.012 (+0.62, 0.237)	+0.018 (+0.83, 0.139)
	GraphCross − Plain	+0.016 (+0.85, 0.131)	+0.009 (+0.46, 0.366)	+0.012 (+0.49, 0.333)	+0.010 (+0.80, 0.147)
	NodeCross − NodeRes	+0.006 (+0.26, 0.588)	+0.009 (+0.69, 0.195)	-0.001 (-0.13, 0.787)	+0.003 (+0.33, 0.503)
	GraphCross − GraphRes	-0.003 (-0.30, 0.545)	+0.002 (+0.08, 0.874)	+0.000 (+0.00, 0.999)	-0.008 (-0.32, 0.513)
	GraphRes − NodeRes	+0.015 (+0.87, 0.125)	-0.003 (-0.27, 0.583)	-0.003 (-0.18, 0.714)	+0.005 (+0.23, 0.637)
	NodeCross − GraphCross	-0.005 (-0.35, 0.476)	+0.010 (+0.50, 0.324)	+0.001 (+0.10, 0.832)	+0.007 (+0.49, 0.334)

Guohao Li, Elias Muller, Abbas Thabet, and Bernard Ghanem. Deeper insights into graph convolutional networks for semi-supervised learning. *arXiv preprint arXiv:1809.02584*, 2018.

Christopher Morris, Roger Wattenhofer, and Kristian Kersting. Tudataset: A collection of benchmark datasets for learning with graphs. *arXiv preprint arXiv:2007.08663*, 2020.

Kenta Oono and Taiji Suzuki. Simple and deep graph convolutional networks. *arXiv preprint arXiv:2006.13604*, 2020.

Yu Rong, Wenbing Huang, Tingyang Xu, and Junzhou Huang. Dropedge: Towards deep graph convolutional networks on node classification. *arXiv preprint arXiv:1907.10903*, 2019.

Jake Topping, Francesco Di Giovanni, Benjamin Chamberlain, Michael Bronstein, Douwe Vendrig, and Pietro Lio. Understanding over-squashing and bottlenecks on graphs via curvature. *arXiv preprint arXiv:2111.14522*, 2021.

Zonghan Wu, Shirui Pan, Fengwen Chen, Guoding Long, Cheng Zhang, and Sheng Yu Zhang. A comprehensive survey on graph neural networks. *IEEE Transactions on Neural Networks and Learning Systems*, 32(1):4–24, 2021.

Keyulu Xu, Chengtao Li, Yonglong Tian, Xiao Du, Jure Leskovec, and Stefanie Jegelka. Representation learning on graphs with jumping knowledge networks. In *International Conference on Machine Learning*, pages 5449–5458, 2018.

Rex Ying, Jiaxuan You, Christopher Morris, Xiang Ren, William L Hamilton, and Jure Leskovec. Hierarchical graph representation learning with differentiable pooling. In *Advances in neural information processing systems*, pages 4800–4810, 2018.

Lingxiao Zhao and Leman Akoglu. Pairnorm: Tackling over-smoothing in gns. *arXiv preprint arXiv:2009.10698*, 2020.

Jie Zhou, Ganqu Cui, Zhengyan Zhang, Cheng Yang, Zhaocheng Liu, Zhongyuan Wang, Peng Li, Ming Sun, Yu Shi, et al. Graph neural networks: A review of methods and applications. *AI Open*, 1:57–81, 2020.